

RAJALAKSHMI ENGINEERING COLLEGE

CS23333
Object Oriented Programming Using JAVA

Contents

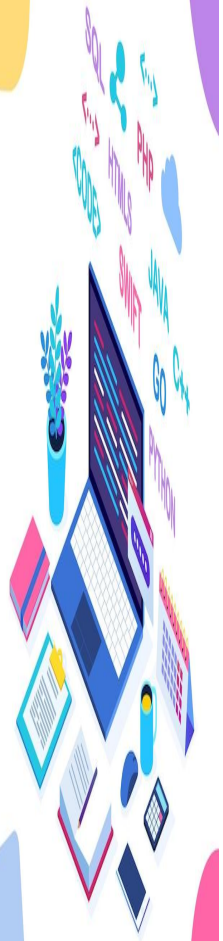
UNIT-II CLASSES AND INHERITANCE

01 Classes and Objects

02 Constructors

03 Inheritance

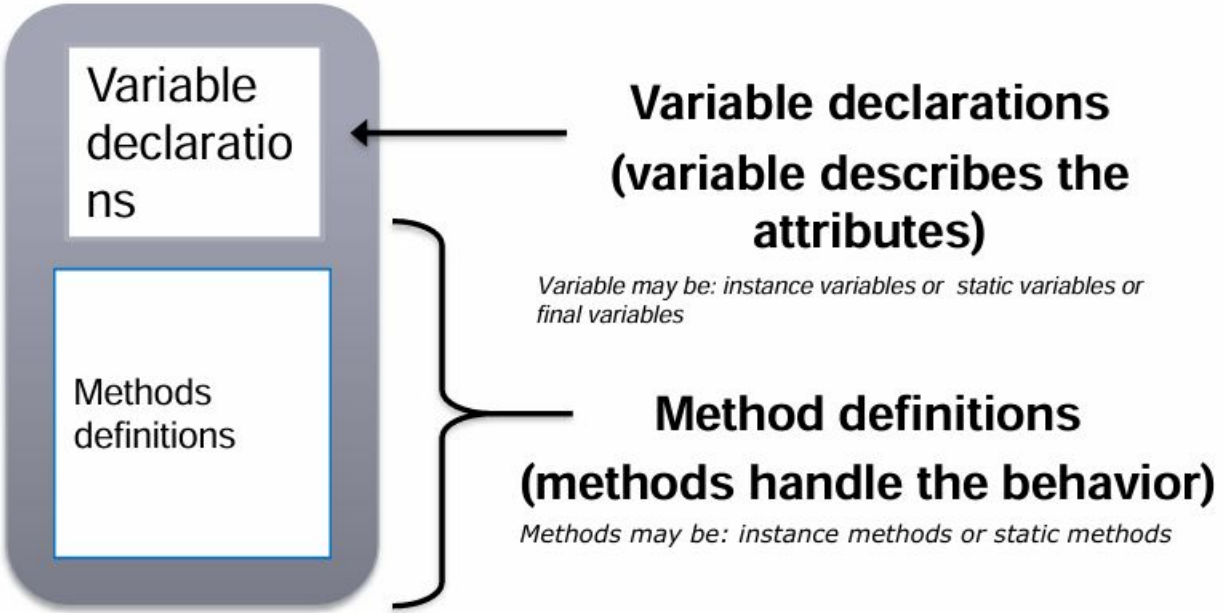
04 Method Overriding

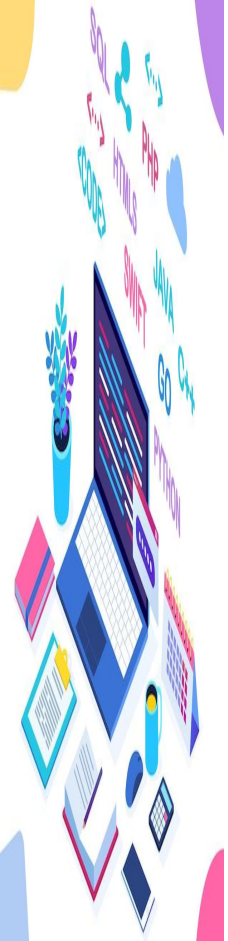


Classes and Objects

Class

- A class contains variable declarations and method definitions.

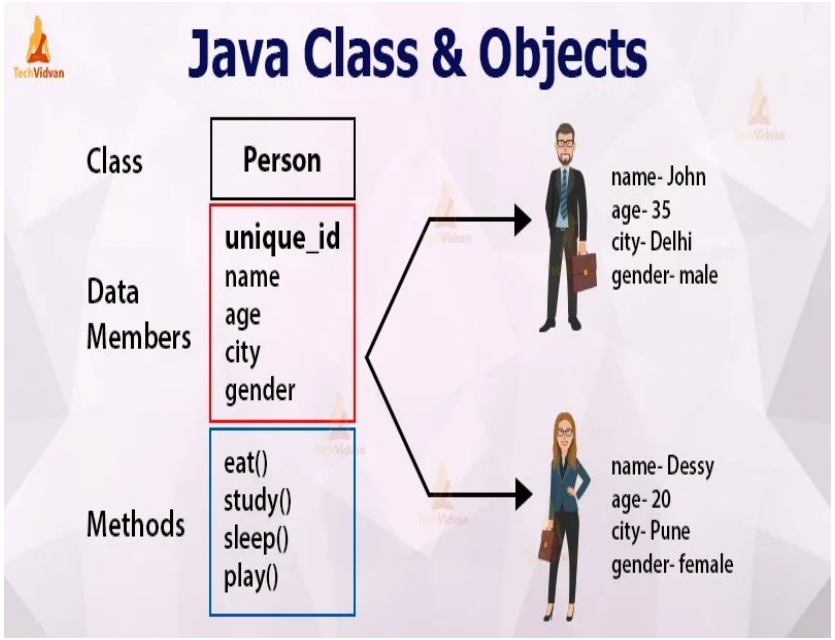


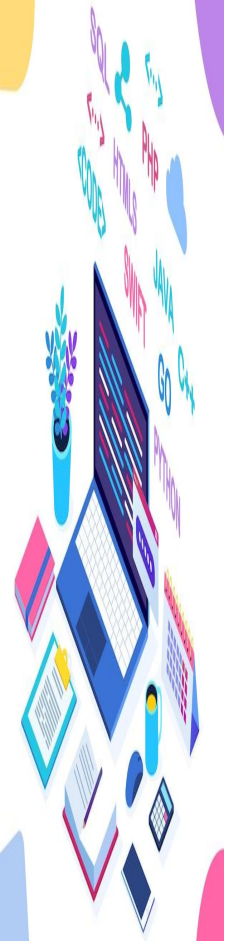


Classes and Objects

Class

- A class is a group of objects which have common properties.
- It is a template or blueprint from which objects are created.
- It is a logical entity.

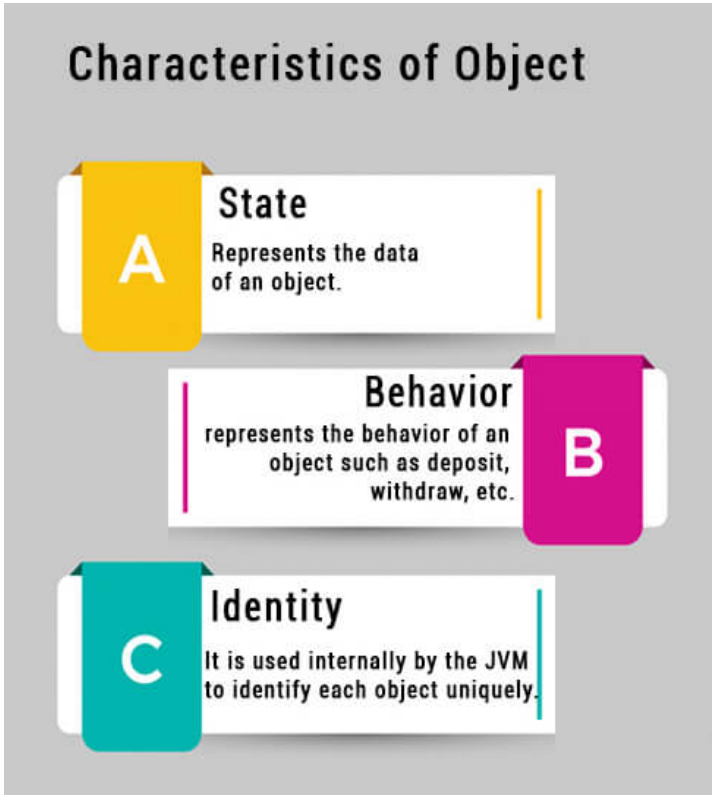


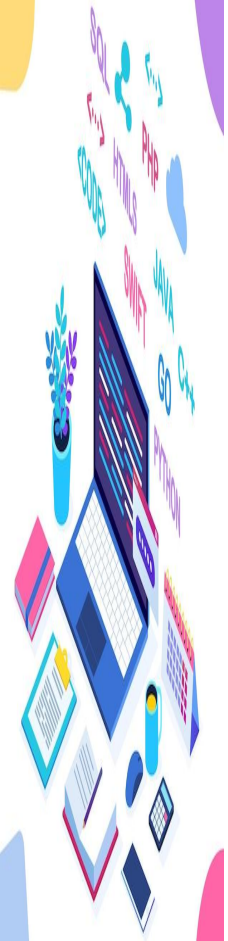


Classes and Objects

Object

- A Java object is a member (also called an instance) of a Java class.
- Each object has an identity, a behavior and a state. The state of an object is stored in fields (variables), while methods (functions) display the object's behavior.
- Objects are created at runtime from templates, which are also known as classes.





Classes and Objects

General form

```
class classname
{
 type instance-variable1;
// ...
 type instance-variableN;
 type methodname1(parameter-list)
{
      // body of method
 }
 type methodname2(parameter-list)
{
      // body of method
 }
 // ...
 type methodnameN(parameter-list)
{
      // body of method
 }
}
```



Classes and Objects

A Simple Class

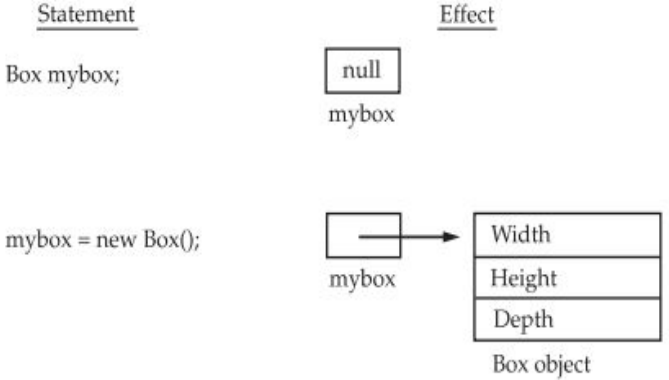
```
class Box {
    double width;
    double height;
    double depth;
}
```

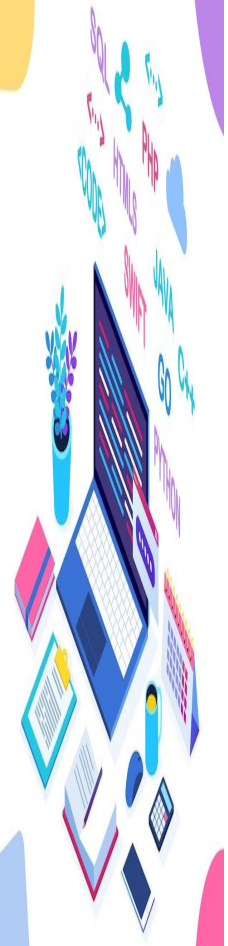
Declaring Objects

```
Box mybox = new Box();
```

Or

```
Box mybox; // declare reference to object
mybox = new Box(); // allocate a Box object
```

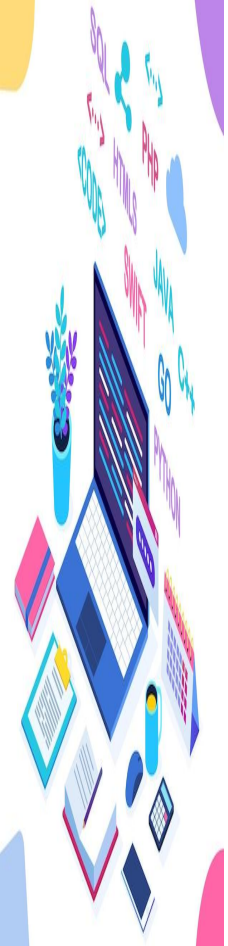




Classes and Objects

A Simple Class

```
class BoxDemo
{
 public static void main(String args[])
 {
        Box mybox = new Box();
        double vol;
        // assign values to mybox's instance variables
        mybox.width = 10;
        mybox.height = 20;
        mybox.depth = 15;
        // compute volume of box
        vol = mybox.width * mybox.height * mybox.depth;
        System.out.println("Volume is " + vol);
 }
}
```



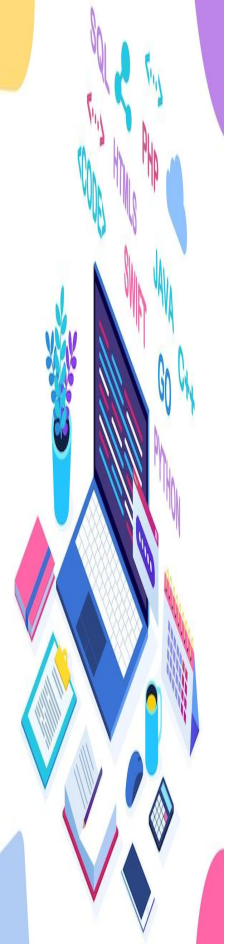
Introducing Methods

Methods is a large one because Java gives them so much power and flexibility.

This is the general form of a method:

```
type name(parameter-list)
{
    // body of method
}
```

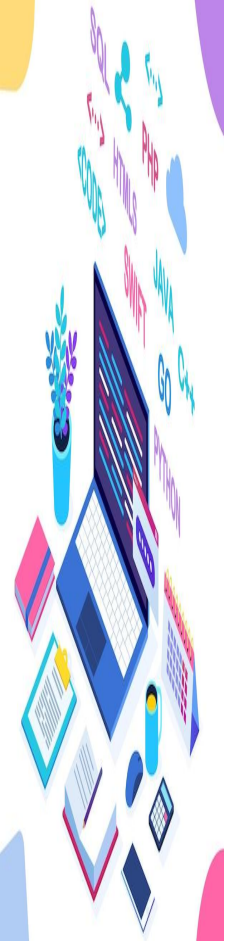
Introducing Methods



```
class Box
{
 double width;
 double height;
 double depth;
double volume()
 {
        return width * height * depth;
}
}
```

```
class BoxDemo {
 public static void main(String args[])  {
        Box b = new Box();
        double vol;
// assign values to b's instance variables
        b.width = 10;
        b.height = 20;
        b.depth = 15;
// call volume method
        double vol=b.volume();
        System.out.println("Volume is"+vol);
 }
}
```

Introducing Methods



```
class Box
{
 double width;
 double height;
 double depth;
}
void setDim(int w, int h, int d)
{
        width=w;
        height=h;
        depth=d;
}
double volume()
{
     return width * height * depth;
}
}
```

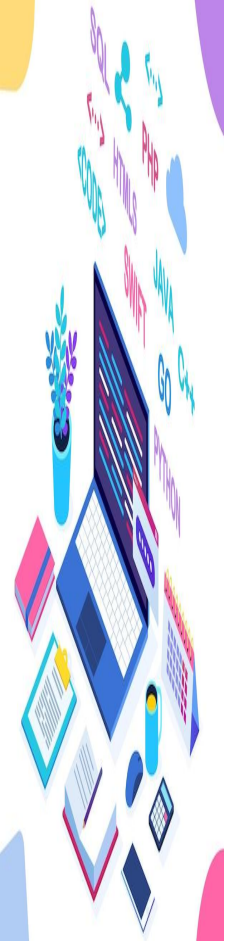
```
class BoxDemo {
 public static void main(String args[])  {
        Box b1 = new Box();
        Box b2 = new Box();
        double vol1,vol2;
// initialize each box
        b1. setDim(10,20,15);
        b2. setDim(3,6,9);
// call volume method
       double vol1=b1.volume();
       System.out.println("Volume is"+vol1);
       double vol2=b2.volume();
       System.out.println("Volume is"+vol2);
}
}
```

Constructors



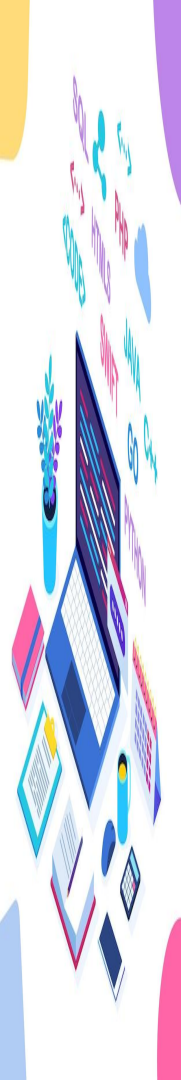
- While designing a class, the class designer can define within the class, a special method called ‘constructor’
- Constructor is automatically invoked whenever an object of the class is created
- Rules to define a constructor
  - A constructor has the same name as the class name
  - A constructor should not have a return type
  - A constructor can be defined with any access specifier (like private, public)
  - A class can contain more than one constructor, So it can be overloaded

Constructors



```
class Box {
  double width;
  double height;
  double depth;
  // This is the constructor for Box.
  Box() {
    System.out.println("Constructing Box");
    width = 10;
    height = 10;
    depth = 10;
  }
  // compute and return volume
  double volume() {
    return width * height * depth;
  }
}
```

```
class BoxDemo6 {
  public static void main(String args[]) {
    // declare, allocate, and initialize Box objects
    Box b1 = new Box();
    Box b2 = new Box();
    double vol1,vol2;
    // get volume of first box
    vol1 = b1.volume();
    System.out.println("Volume is " + vol1);
    // get volume of second box
    vol2 = b2.volume();
    System.out.println("Volume is " + vol2);
  }
}
```



Parameterized Constructors

```
class Box {
  double width;
  double height;
  double depth;
  // This is the constructor for Box.
  Box(double w, double h, double d) {
    width = w;
    height = h;
    depth = d;
  }
  // compute and return volume
  double volume() {
    return width * height * depth;
  }
}
```

```
class BoxDemo7 {
  public static void main(String args[]) {
    // declare, allocate, and initialize Box objects
    Box b1 = new Box(10, 20, 15);
    Box b2 = new Box(3, 6, 9);
    double vol1,vol2;
    // get volume of first box
    vol1 = b1.volume();
    System.out.println("Volume is " + vol1);
    // get volume of second box
    vol2 = b2.volume();
    System.out.println("Volume is " + vol2);
  }
}
```

Overloading Constructors

```
class Box {
  double width, height, depth;
  // constructor used when all dimensions specified
Box(double w, double h, double d) {
    width = w;
    height = h;
    depth = d;
}
Box() { // constructor used when no dimensions specified
    width = -1;  // use -1 to indicate
    height = -1; // an uninitialized
    depth = -1;  // box
}
Box(double len) { // constructor used when cube is created
    width = height = depth = len;
}
double volume() {  // compute and return volume
    return width * height * depth;
  }
}
```

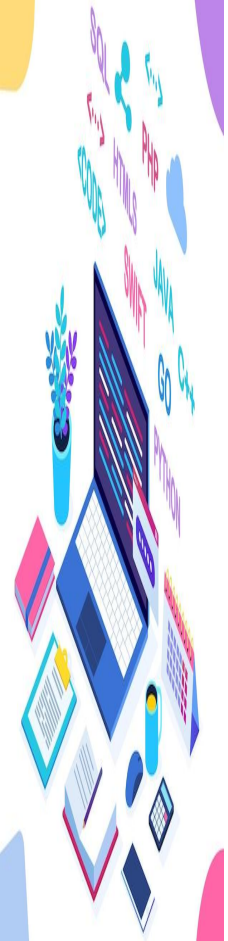
```
class OverloadCons {
  public static void main(String args[]) {
    // create boxes using the various constructors
    Box b1 = new Box(10, 20, 15);
    Box b2 = new Box();
    Box mycube = new Box(7);
    double vol1, vol2,vol3;
    // get volume of first box
    vol1 = b1.volume();
    System.out.println("Volume of b1 is " + vol1);
    // get volume of second box
    vol2 = b2.volume();
    System.out.println("Volume of b2 is " + vol2);
// get volume of cube
    vol3 = mycube.volume();
    System.out.println("Volume of mycube is " + vol3);
  }
}
```

Using Objects as Parameters

```
class Box {
  double width, height, depth;
Box(Box ob) { // pass object to constructor
    width = ob.width;
    height = ob.height;
    depth = ob.depth;
  }
// constructor used when all dimensions specified
Box(double w, double h, double d) {
    width = w;
    height = h;
    depth = d;
}
Box() { // constructor used when no dimensions specified
    width = -1;  // use -1 to indicate
    height = -1; // an uninitialized
    depth = -1;  // box
}
Box(double len) { // constructor used when cube is created
    width = height = depth = len;
}
double volume() {  // compute and return volume
    return width * height * depth;
 }}
```

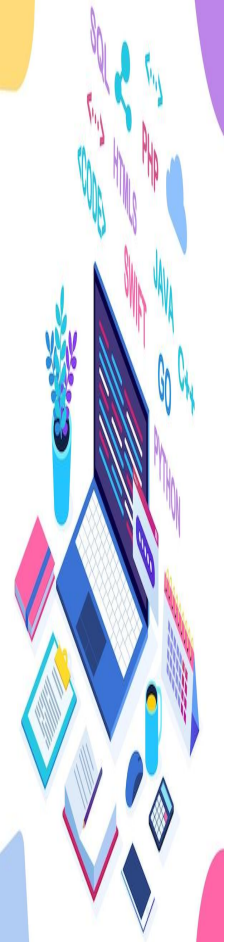
```
class OverloadCons {
  public static void main(String args[]) {
    // create boxes using the various constructors
    Box b1 = new Box(10, 20, 15);
    Box b2 = new Box();
    Box mycube = new Box(7);
 Box myclone = new Box(mybox1); // create copy of mybox1
    double vol1, vol2,vol3,vol4;
    // get volume of first box
    vol1 = b1.volume();
    System.out.println("Volume of b1 is " + vol1);
    // get volume of second box
    vol2 = b2.volume();
    System.out.println("Volume of b2 is " + vol2);
// get volume of cube
    vol3 = mycube.volume();
    System.out.println("Volume of mycube is " + vol3);
vol4 = myclone.volume();
System.out.println("Volume of clone is " + vol4);
  } }
```

Inheritance

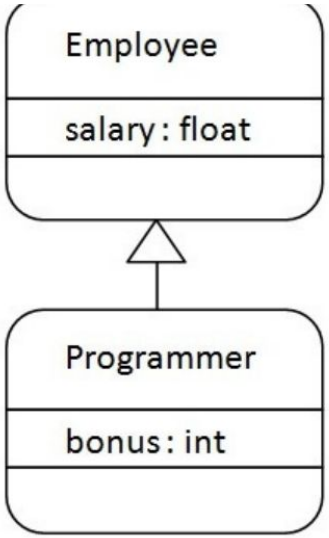


- The mechanism of deriving a new class from the existing class is called inheritance.
- The existing class is called base class, super class, parent class. **(a class that is inherited)**
- The new class is called derived class, subclass, child class **(The class that does the inheriting)**
- The inheritance allows subclasses to inherit all the variables and methods of the parent classes.
- To inherit a class, you simply incorporate the definition of one class into another by using the extends keyword.

Java Inheritance Example



- Programmer is the subclass and Employee is the superclass.
- The relationship between the two classes is Programmer IS-A Employee. It means that Programmer is a type of Employee.



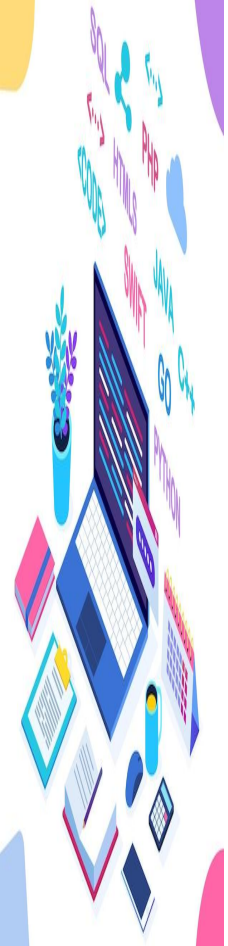
Java Inheritance Example

```
class Employee{
 float salary=40000;
}
class Programmer extends Employee{
 int bonus=10000;

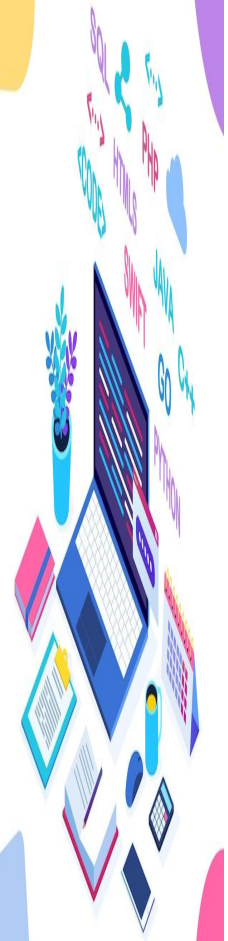
public static void main(String args[]){
   Programmer p=new Programmer();
   System.out.println("Programmer salary is:"+p.salary);
   System.out.println("Bonus of Programmer is:"+p.bonus);
}
}
```

```
Programmer salary is:40000.0
Bonus of programmer is:10000
```

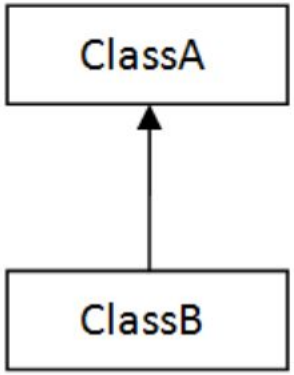
Types of inheritance in java



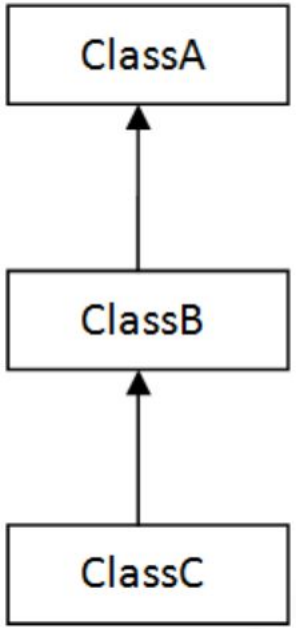
- On the basis of class, there can be three types of inheritance in java: single, multilevel and hierarchical.
- In java programming, multiple and hybrid inheritance is supported through interface



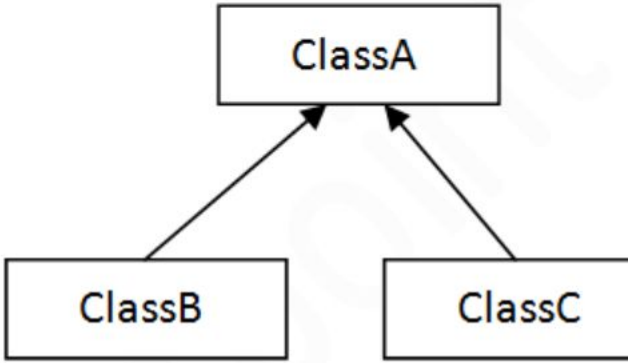
Types of inheritance in java



1) Single



2) Multilevel



3) Hierarchical

Single Inheritance Example

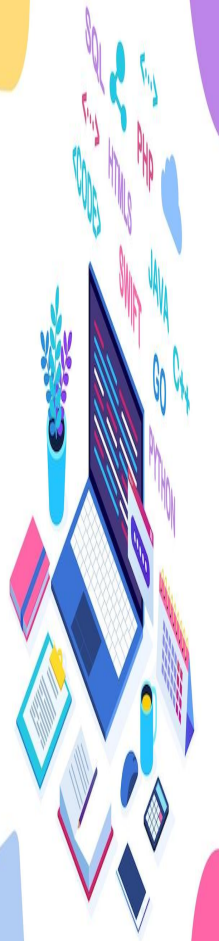


```
class Animal{
void eat(){System.out.println("eating...");}
}
class Dog extends Animal{
void bark(){System.out.println("barking...");}
}
class TestInheritance{
public static void main(String args[]){
Dog d=new Dog();
d.bark();
d.eat();
}}
```

Output:

```
barking...
eating...
```

Multilevel Inheritance Example

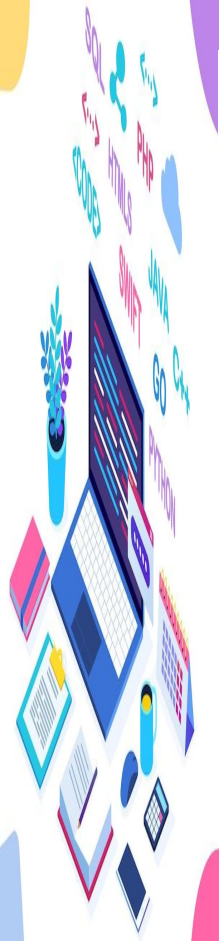


```
class Animal{
void eat(){System.out.println("eating...");}
}
  class Dog extends Animal{
  void bark(){System.out.println("barking...");}
  }
  class BabyDog extends Dog{
  void weep(){System.out.println("weeping...");}
  }
  class TestInheritance2{
  public static void main(String args[]){
  BabyDog d=new BabyDog();
  d.weep();
  d.bark();
  d.eat();
  }}
```

Output:

```
weeping...
barking...
eating...
```

Hierarchical Inheritance Example



```
class Animal{
void eat(){System.out.println("eating...");}
}
class Dog extends Animal{
void bark(){System.out.println("barking...");}
}
class Cat extends Animal{
void meow(){System.out.println("meowing...");}
}
class TestInheritance3{
public static void main(String args[]){
Cat c=new Cat();
c.meow();
c.eat();
//c.bark();//C.T.Error
}}
```

Output:

```
meowing...
eating...
```

Inheritance

```
// Create a superclass.
class A {
  int i, j;
  void showij() {
    System.out.println("i and j: " + i + " " + j);
  }
}
// Create a subclass by extending class A.
class B extends A {
  int k;
  void showk() {
    System.out.println("k: " + k);
  }
void sum() {
    System.out.println("i+j+k: " + (i+j+k));
  }
}
```

```
class SimpleInheritance {
  public static void main(String args []) {
    A superOb = new A();
    B subOb = new B();
    // The superclass may be used by itself.
    superOb.i = 10;
    superOb.j = 20;
    System.out.println("Contents of superOb: ");
    superOb.showij();
// The subclass has access to all public members of its superclas
    subOb.i = 7;
    subOb.j = 8;
    subOb.k = 9;
    System.out.println("Contents of subOb: ");
    subOb.showij();
    subOb.showk();
    System.out.println("Sum of i, j and k in subOb:");
    subOb.sum();
  }
}
```

Member Access and Inheritance

Although a subclass includes all of the members of its superclass, it cannot access those members of the superclass that have been declared as private.

```
/* In a class hierarchy, private members remain
   private to their class.
   This program contains an error and will not
   compile.
*/
// Create a superclass.
class A {
  int i; // public by default
  private int j; // private to A
  void setij(int x, int y) {
    i = x;
    j = y;
  }
}
// A's j is not accessible here.
class B extends A {
  int total;
  void sum() {
    total = i + j; // ERROR, j is not accessible here
  }
}
```

```
class Access {
  public static void main(String args[]) {
    B subOb = new B();
    subOb.setij(10, 12);
    subOb.sum();
    System.out.println("Total is " + subOb.total);
  }
}
```

This program will not compile because the use of j inside the sum( ) method of B causes an access violation. Since j is declared as private, it is only accessible by other members of its own class. Subclasses have no access to it.

NOTE: A class member that has been declared as private will remain private to its class. It is not accessible by any code outside its class, including subclasses.

Inheritance

```
class Box {
  double width, height, depth;
Box(double w, double h, double d) {
    width = w;
    height = h;
    depth = d;
}
double volume() {  // compute and return
volume
    return width * height * depth;
 }
}
```

```
// Here, Box is extended to include weight.
class BoxWeight extends Box {
double weight; // weight of box
  // constructor for BoxWeight
  BoxWeight(double w, double h, double d,
double m) {
    width = w;
    height = h;
    depth = d;
    weight = m;
  }
}
```

Inheritance

```
class DemoBox {
  public static void main(String args[]) {
    // create boxes using the various constructors
     BoxWeight b1 = new  BoxWeight(10, 20, 15, 34.3);
     BoxWeight b2 = new  BoxWeight(2, 3, 4, 0.076);

    double vol1, vol2,vol3,vol4;
    // get volume of first box
    vol1 = b1.volume();
    System.out.println("Volume of b1 is " + vol1);
System.out.println("Weight of b1 is " + b1.weight);
    // get volume of second box
    vol2 = b2.volume();
    System.out.println("Volume of b2 is " + vol2);
System.out.println("Weight of b2 is " + b2.weight);
```

Output:

Volume of b1 is 3000.0
Weight b1 is 34.3
Volume of b2 is 24.0
Weight of b2 is 0.076

Using super to Call Superclass Constructors

A subclass can call a constructor defined by its superclass by use of the following form of super:

super(arg-list);

```
// BoxWeight now uses super to initialize its Box attributes.
class BoxWeight extends Box {
  double weight; // weight of box
  // initialize width, height, and depth using super()
  BoxWeight(double w, double h, double d, double m) {
    super(w, h, d); // call superclass constructor
    weight = m;
  }
}
```

Here, BoxWeight( ) calls super( ) with the arguments w, h, and d. This causes the Box constructor to be called, which initializes width, height, and depth using these values. BoxWeight no longer initializes these values itself. It only needs to initialize the value unique to it: weight. This leaves Box free to make these values private if desired. In the preceding example, super( ) was called with three arguments. Since constructors can be overloaded, super( ) can be called using any form defined by the superclass. The constructor executed will be the one that matches the arguments.

A Second Use for super

The second form of super acts somewhat like this, except that it always refers to the superclass of the subclass in which it is used. This usage has the following general form:

super.member

Here, member can be either a method or an instance variable. This second form of super is most applicable to situations in which member names of a subclass hide members by the same name in the superclass.

Although the instance variable i in B hides the i in A, super allows access to the i defined in the superclass. As you will see, super can also be used to call methods that are hidden by a subclass.

```
// Using super to overcome name hiding.
class A {
  int i;
}
// Create a subclass by extending class A.
class B extends A {
  int i; // this i hides the i in A
  B(int a, int b) {
    super.i = a; // i in A
    i = b; // i in B
  }
  void show() {
    System.out.println("i in superclass: " + super.i);
    System.out.println("i in subclass: " + i);
  }
}
class UseSuper {
  public static void main(String args[]) {
    B subOb = new B(1, 2);
    subOb.show();
  }
}
```

This program displays the following:

```
i in superclass: 1
i in subclass: 2
```



Method Overriding

In a class hierarchy, when a method in a subclass has the same name and type signature as a method in its superclass, then the method in the subclass is said to override the method in the superclass. When an overridden method is called from within its subclass, it will always refer to the version of that method defined by the subclass. The version of the method defined by the superclass will be hidden.

Method Overriding

```
// Method overriding.
class A {
  int i, j;
  A(int a, int b) {
    i = a;
    j = b;
  }
  // display i and j
  void show() {
    System.out.println("i and j: " + i + " " + j);
  }
}
class B extends A {
  int k;
  B(int a, int b, int c) {
    super(a, b);
    k = c;
  }
  // display k – this overrides show() in A
  void show() {
 System.out.println("k: " + k);
  }
}
```

```
class Override {
  public static void main(String args[]) {
    B subOb = new B(1, 2, 3);
   subOb.show(); // this calls show() in B
  }
}
```

The output produced by this program is shown here: k: 3

When show( ) is invoked on an object of type B, the version of show( ) defined within B is used. That is, the version of show( ) inside B overrides the version declared in A.

Method Overriding

If you wish to access the superclass version of an overridden method, you can do so by using super. For example, in this version of B, the superclass version of show( ) is invoked within the subclass' version. This allows all instance variables to be displayed.

```
class B extends A {
  int k;
  B(int a, int b, int c) {
    super(a, b);
    k = c;
  }
  void show() {
    super.show(); // this calls A's show()
    System.out.println("k: " + k);
  }
}
```

If you substitute this version of A into the previous program, you will see the following output:

```
  i and j: 1 2
  k: 3
```

Here, super.show( ) calls the superclass version of show( )

Method Overriding



Method overriding occurs only when the names and the type signatures of the two methods are identical. If they are not, then the two methods are simply overloaded.

Method Overloaded

```
// Methods with differing type signatures are overloaded
– not overridden.
class A {
  int i, j;
  A(int a, int b) {
    i = a;
    j = b;
  }
    void show() {  // display i and j
    System.out.println("i and j: " + i + " " + j);
  }
}
// Create a subclass by extending class A.
class B extends A {
  int k;
  B(int a, int b, int c) {
    super(a, b);
    k = c;
  }
  // overload show()
  void show(String msg) {
    System.out.println(msg + k);
  } }
```

```
class Override {
  public static void main(String args[]) {
    B subOb = new B(1, 2, 3);
    subOb.show("This is k: "); // this calls show() in B
    subOb.show(); // this calls show() in A
  }
}
```

The output produced by this program is shown here: This is k: 3 i and j: 1 2 The version of show( ) in B takes a string parameter. This makes its type signature different from the one in A, which takes no parameters. Therefore, no overriding (or name hiding) takes place. Instead, the version of show( ) in B simply overloads the version of show( ) in A.

The Object Class

There is one special class, Object, defined by Java. All other classes are subclasses of Object. That is, Object is a superclass of all other classes. This means that a reference variable of type Object can refer to an object of any other class. Also, since arrays are implemented as classes, a variable of type Object can also refer to any array. Object defines the following methods, which means that they are available in every object

Method	Purpose
Object clone( )	Creates a new object that is the same as the object being cloned.
boolean equals(Object *object*)	Determines whether one object is equal to another.
void finalize( )	Called before an unused object is recycled.
Class<?> getClass( )	Obtains the class of an object at run time.
int hashCode( )	Returns the hash code associated with the invoking object.
void notify( )	Resumes execution of a thread waiting on the invoking object.
void notifyAll( )	Resumes execution of all threads waiting on the invoking object.
String toString( )	Returns a string that describes the object.
void wait( ) void wait(long *milliseconds*) void wait(long *milliseconds*, int *nanoseconds*)	Waits on another thread of execution.

The Object Class



The methods getClass( ), notify( ), notifyAll( ), and wait( ) are declared as final. You may override the others. These methods are described elsewhere in this book. However, notice two methods now: equals( ) and toString( ). The equals( ) method compares two objects. It returns true if the objects are equal, and false otherwise. The precise definition of equality can vary, depending on the type of objects being compared. The toString( ) method returns a string that contains a description of the object on which it is called. Also, this method is automatically called when an object is output using println( ). Many classes override this method. Doing so allows them to tailor a description specifically for the types of objects that they create.

Classes and Objects

- A class contains members which can either be variables(fields) or methods(behaviors).
- A variable declared within a class(outside any method) is known as an instance variable.
- A variable declared within a method is known as local variable.
- Variables with method declarations are known as parameters or arguments.
- A class variable can also be declared as static where as a local variable cannot be static.

The new operator,

Dynamically allocates memory for an object

Creates the object on the heap

Returns a reference to it

The reference is then stored in the variable

- Use of this keyword
  - this can be used to refer instance variables when there is a clash with local variables or method arguments
  - this can be used to call overloaded constructors from another constructor of the same class
- Use this.variableName to explicitly refer to the instance variable.
- Use variable Name to refer to the parameter.
- The this reference is implicitly used to refer to instance variables and methods.
- It CANNOT be used in a static method.
  -

```
class Test {
    private int value;
    Test(int value) {
        this.value = value;
    }
    public int getValue() {
        return value;
    }
}
public class Main {
    public static void main(String[] args) {
        Test obj = new Test(10);
        System.out.println(obj.value);
    }
}
```

OUTPUT:

```
Main.java:14: error: value has private access in Test
        System.out.println(obj.value);
                           ^
1 error
```

```
class Box {
    int height;
    Box(int height) {
        this.height = height;
    }
    void modifyHeight(Box b) {
        b.height += 10;
    }
}
public class Main {
    public static void main(String[] args) {
        Box b1 = new Box(20);
        b1.modifyHeight(b1);
        System.out.println(b1.height);
    }
}
```

OUTPUT

30

```
class Person {
    String name;
    void setName(String n) {
        name = n;
    }
    void printName() {
        System.out.println(name);
    }
}

class Test {
    public static void main(String[] args) {
        Person p = new Person();
        p.printName();
    }
}
```

OUTPUT

null

```
class Sample {
    int x = 10;

    void display() {
        System.out.println("x = " + x);
    }

    public static void main(String[] args) {
        Sample s = new Sample();
        s.display();
    }
}
```

OUTPUT:

x = 10

```
class Box {
    int length = 5;
    int width = 4;

    int area() {
        return length * width;
    }

    public static void main(String[] args) {
        Box b = new Box();
        System.out.println("Area = " + b.area());
    }
}
```

OUTPUT:

20

```
class A {
    int x = 50;
}

public class Main {
    public static void main(String[] args) {
        A obj1 = new A();
        A obj2 = obj1;
        obj2.x = 100;
        System.out.println(obj1.x);
    }
}
```

OUTPUT:

100

```
class A {
    int y = 30;
}

public class Main {
    public static void main(String[] args) {
        A a1 = new A();
        A a2 = new A();
        a1.y = 50;
        System.out.println(a2.y);
    }
}
```

OUTPUT:

30

```
class A {
    int p = 5;
    int q = 2;
}

class Main {
    public static void main(String[] args) {
        A obj = new A();
        System.out.println(obj.p + obj.q);
    }
}
```

OUTPUT:

7

```
class A {
    int val = 20;
}

public class Main {
    public static void main(String[] args) {
        A obj1 = new A();
        A obj2 = obj1;
        obj2.val += 5;
        System.out.println(obj1.val);
    }
}
```

OUTPUT:

25

```
class Person {
    int age = 18;
}

public class Main {
    public static void main(String[] args) {
        Person p = new Person();
        p.age += 2;
        System.out.println("Age: " + p.age);
    }
}
```

OUTPUT:

Age: 20

```
class Ball {
    int size = 11;
}

class Game {
    public static void main(String[] args) {
        Ball b1 = new Ball();
        Ball b2 = new Ball();
        b2.size = 10;
        System.out.println(b1.size);
    }
}
```

OUTPUT:

11

```
class Demo {
    void printMessage() {
        System.out.println("Hello from Demo");
    }
}

public class Main {
    public static void main(String[] args) {
        Demo d = new Demo();
        d.printMessage();
    }
}
```

OUTPUT:

Hello from Demo

```
class Box {
    int volume(int l, int b, int h) {
        return l * b * h;
    }
}

public class Main {
    public static void main(String[] args) {
        Box b = new Box();
        System.out.println(b.volume(2, 3, 4));
    }
}
```

OUTPUT:

24

```
class Alpha {
    void greet(String name) {
        System.out.println("Hello " + name);
    }
}

public class Main {
    public static void main(String[] args) {
        Alpha obj = new Alpha();
        obj.greet("Anu");
    }
}
```

OUTPUT:

Hello Anu

```
class MathUtils {
    int add(int x) {
        return x + x;
    }
}

public class Main {
    public static void main(String[] args) {
        MathUtils m = new MathUtils();
        System.out.println(m.add(5));
    }
}
```

OUTPUT:

10

```
class MyClass {
    int value;
    MyClass(int v) {
        value = v;
    }
    void changeValue(MyClass obj) {
        obj.value = 50;
    }
}
class Main1 {
    public static void main(String[] args) {
        MyClass obj1 = new MyClass(5);
        MyClass obj2 = obj1;
        obj1.changeValue(obj2);
        System.out.println(obj1.value);
    }
}
```

OUTPUT:

50

```
class MyClass {
    int a;
    MyClass(int val) {
        a = val;
    }
}
class Main1 {
    public static void main(String[] args) {
        MyClass m1 = new MyClass(5);
        MyClass m2 = m1;
        m2 = new MyClass(15);
        System.out.println(m1.a);
    }
}
```

OUTPUT:

5

```
class Sample {
    int data;
    Sample(int d) {
        data = d;
    }
}
class Main {
    public static void main(String[] args) {
        Sample s1 = new Sample(5);
        Sample s2 = s1;
        s1.data = 15;
        Sample s3 = new Sample(25);
        s2 = s3;
        System.out.println(s1.data);
    }
}
```

OUTPUT:

15

```
class Student {
    String name;
    int age;
    Student(String name, int age) {
        this.name = name;
        this.age = age;
    }
    void display() {
        System.out.println("Name: " + name + ", Age: "
+ age);
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Student student = new Student("Alice", 20);
        student.display();
    }
}
```

OUTPUT:

Name: Alice, Age: 20

```
class Test {
    int x;
    Test(int x) {
        this.x = x;
    }
    void display() {
        System.out.println("Value of x: " + x);
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Test testObj = new Test(10);
        testObj.display();
    }
}
```

OUTPUT:

Value of x: 10

Which of the following statements about the 'this' keyword in Java are false?

- 'this' keyword refers to the current class object.
- 'this' can be used to invoke the current class constructor.
- 'this' can be used to refer to static members of the class.
- 'this' is used to differentiate between instance variables and local variables in a constructor.

What does **this** keyword refer to in Java?

It refers to the current instance of the class

It refers to the superclass of the current class

It refers to the current static method

It refers to the current object's superclass instance

In Java, where can you use **this** keyword?

- Only inside static methods
- Only inside instance methods
- Only inside constructors
- Inside both instance methods and constructors

```
public class Main
{
int a;
public Main(int a)
{
this.a = a;
}
public static void main(String[] args)
{
Main obj = new Main(87);
System.out.print(obj.a);
}
}
```

OUTPUT:

87

```
class Demo {
    int value;
    Demo(int v) {
        value = v;
    }
}
public class Main {
    public static void main(String[] args) {
        Demo d1 = new Demo(5);
        Demo d2 = d1;
        d2.value = 15;
        System.out.println(d1.value);
    }
}
```

OUTPUT:

15

```
class Sample {
    String msg;
    Sample(String m) {
        msg = m;
    }
    void show() {
        System.out.println(msg);
    }
}
public class Main {
    public static void main(String[] args) {
        Sample s1 = new Sample("Java");
        Sample s2 = new Sample("Python");
        s1 = s2;
        s1.show();
    }
}
```

OUTPUT:

Python

```
class Test {
    Test() {
        System.out.println("Default constructor");
    }
    Test(int x) {
        System.out.println("Parameterized constructor: " + x);
    }
}
public class Main {
    public static void main(String[] args) {
        Test t = new Test(50);
    }
}
```

OUTPUT:

Parameterized constructor: 50

```
class A {
    int x = 10;
}
public class Main {
    public static void main(String[] args) {
        A obj1 = new A();
        A obj2 = obj1;
        obj2.x = 25;
        System.out.println(obj1.x);
    }
}
```

OUTPUT: 25

```
class B {
   int y;
        B(int y)
        {
        this.y = y;
        }
}
public class Main {
   public static void main(String[] args) {
      B b1 = new B(5);
      B b2 = new B(15);
      b1 = b2;
      System.out.println(b1.y);
   }
}
```

OUTPUT:

15

Q3.
When two references point to the same object in Java:
A) A copy of the object is created
B) Both references share the same memory location
C) The object is cloned automatically
D) Java throws an exception

What is **true** about objects in Java?
A) Objects are stored in Stack
B) Objects are always stored in Heap memory
C) Objects cannot be reassigned
D) Objects are destroyed immediately after use

Q6.
Which is **true** about constructors?
A) They have the same name as the class
B) They must return an int
C) They cannot be overloaded
D) They are always private

Which of the following is **not correct** about this keyword?
A) Refers to current instance
B) Used to call another constructor
C) Can be used inside static methods
D) Helps differentiate local and instance variables

```
class Demo {
    int z;
}
public class Main {
    public static void main(String[] args) {
        Demo d1 = new Demo();
        Demo d2 = d1;
        d1 = null;
        System.out.println(d2.z);
    }
}
```

Output?
A) 0
B) NullPointerException
C) GarbageCollector error
D) Compilation Error

```
class Test {
    int num;
    Test(int n) {
        this.num = n;
    }
}
public class Main {
    public static void main(String[] args) {
        Test t = new Test(50);
        System.out.println(t.num);
    }
}
```

OUTPUT:

50

```
public class Main {
    public static void main(String[] args) {
        new A(10);
    }
}
class A {
    A()
    {
        System.out.println("Default");
    }
    A(int x)
    {
        this();
        System.out.println(x);
    }
}
```

Output?
A) 10
B) Default
C) Default 10
D) Error

```
class Test {
    int x;
}
public class Main {
    public static void main(String[] args) {
        Test t = new Test();
        System.out.println(t.x);
    }
}
```

What will it print?
A) 0
B) null
C) Compilation Error
D) Garbage Value